08 — HuggingFace Ecosystem

Time: ~4-5 hours | **Level**: Advanced

What you'll learn:

- HuggingFace Hub: model/dataset repository
- `transformers` library: load any pre-trained model in 2 lines
- Tokenisers: BPE in practice (AutoTokenizer)
- Pipelines: zero-code inference
- `datasets` library: efficient data loading
- Model architectures: AutoModel, AutoModelForCausalLM
- Generation: temperature, top-k, top-p sampling

Prerequisites: Notebooks 05-07 (NLP, attention, transformers)

Why HuggingFace?

In Notebook 07, we built a Transformer from scratch. In practice, you'd never do that. HuggingFace provides:

- 500,000+ pre-trained models
- Standardised API for all model architectures
- Tokenisers, datasets, training utilities
- The community standard for NLP/LLM work

In [ ]:
```
# ── Install (run once) ──────────────────────────────────────
# !pip install transformers datasets tokenizers accelerate -q

import torch
from transformers import AutoTokenizer, AutoModelForCausalLM, AutoModel
from transformers import pipeline
import numpy as np

print(f'PyTorch: {torch.__version__}')
print(f'CUDA available: {torch.cuda.is_available()}')
print('\n💡 All code in this notebook works on CPU. GPU just makes it faster
```

1. Tokenisers — BPE in Practice

In Notebook 05, we built a simple BPE. HuggingFace tokenisers are the production version.

Every model has a paired tokeniser trained on the same data.

In [ ]:
```
# ── Load a real tokeniser ──────────────────────────────────────

# GPT-2's tokeniser (BPE, 50257 tokens)
tokenizer = AutoTokenizer.from_pretrained('gpt2')

text = "Patient reports severe anxiety with panic attacks and sleep disrupti

# Tokenise
tokens = tokenizer.tokenize(text)
token_ids = tokenizer.encode(text)

print(f'Original: "{text}"')
print(f'\nTokens ({len(tokens)}): {tokens}')
print(f'Token IDs: {token_ids}')
print(f'\nVocab size: {tokenizer.vocab_size}')

# Decode back
decoded = tokenizer.decode(token_ids)
print(f'\nDecoded: "{decoded}"')
assert decoded == text, 'Tokenisation is lossless!'

print('\n💡 Notice how common words stay whole ("Patient", "reports")')
print('   but less common words get split ("disruption" → ["dis", "ruption"]
```

In [ ]:
```
# ── Tokeniser deep dive ────────────────────────────────────────

# What the tokeniser really returns (for model input)
encoded = tokenizer(
    text,
    return_tensors='pt',       # return PyTorch tensors
    padding=True,              # pad to same length
    truncation=True,           # truncate if too long
    max_length=32,             # max 32 tokens
)

print('tokenizer() output:')
for key, value in encoded.items():
    print(f'  {key}: shape={value.shape}, dtype={value.dtype}')
    print(f'         values={value[0].tolist()[:15]}...')

print(f'\n💡 input_ids: token indices (what the model sees)')
print(f'   attention_mask: 1=real token, 0=padding (tells model what to ignc

# Batch tokenisation
texts = [
    "Patient reports anxiety.",
```

```
    "Feeling much better after starting medication and therapy sessions.",
]
batch = tokenizer(texts, return_tensors='pt', padding=True, truncation=True)
print(f'\nBatch input_ids shape: {batch["input_ids"].shape}')  # (2, max_ler
print(f'Attention mask: {batch["attention_mask"][0].tolist()}')
print(f'                {batch["attention_mask"][1].tolist()}')
```

2. Loading Pre-trained Models

HuggingFace's `Auto` classes automatically detect the right model architecture:

Class	Use Case
`AutoModel`	Raw transformer outputs (embeddings)
`AutoModelForCausalLM`	Text generation (GPT, LLaMA, Phi)
`AutoModelForSequenceClassification`	Text classification
`AutoModelForTokenClassification`	NER, POS tagging
`AutoModelForSeq2SeqLM`	Encoder-decoder (T5, BART)

In [ ]:
```
# ── Load GPT-2 (small, ~500MB) ──────────────────────────────────

model = AutoModelForCausalLM.from_pretrained('gpt2')
model.eval()  # inference mode

print(f'Model type: {type(model).__name__}')
print(f'Parameters: {sum(p.numel() for p in model.parameters()):,}')
print(f'\nArchitecture (high level):')
for name, module in model.named_children():
    param_count = sum(p.numel() for p in module.parameters())
    print(f'  {name}: {type(module).__name__} ({param_count:,} params)')
```

In [ ]:
```
# ── Manual generation (understand what's happening) ─────────────

prompt = "Patient presents with symptoms of"
input_ids = tokenizer.encode(prompt, return_tensors='pt')

print(f'Prompt: "{prompt}"')
print(f'Input IDs: {input_ids[0].tolist()}')

# Forward pass: get logits for every position
with torch.no_grad():
    outputs = model(input_ids)
    logits = outputs.logits  # (1, seq_len, vocab_size)

print(f'\nLogits shape: {logits.shape}')  # (1, 6, 50257)
print(f'Last position logits: probabilities over {logits.shape[-1]} tokens')

# Get top-5 most likely next tokens
next_token_logits = logits[0, -1, :]  # logits for position AFTER "of"
top5_ids = torch.topk(next_token_logits, 5).indices
```

```
top5_probs = torch.softmax(next_token_logits, dim=-1)[top5_ids]

print(f'\nTop 5 next tokens after "{prompt}":')
for tid, prob in zip(top5_ids, top5_probs):
    token = tokenizer.decode(tid)
    print(f'  "{token}" — probability: {prob.item():.3%}')
```

3. Generation Strategies

How do we go from next-token probabilities to full text?

Strategy	Description	Use Case
Greedy	Always pick highest probability	Deterministic, often repetitive
Top-k	Sample from top k tokens	More diverse
Top-p (nucleus)	Sample from smallest set with cumulative prob ≥ p	Best diversity control
Temperature	Scale logits before softmax (< 1 = sharper, > 1 = flatter)	Control randomness
Beam search	Track top-n sequences in parallel	High quality, slow

In [ ]:
```
# ── Generation with different strategies ──────────────────────

tokenizer.pad_token = tokenizer.eos_token
prompt = "Patient presents with symptoms of"
input_ids = tokenizer.encode(prompt, return_tensors='pt')

print(f'Prompt: "{prompt}"\n')

# Greedy (deterministic)
greedy_output = model.generate(
    input_ids, max_new_tokens=30,
    do_sample=False,  # greedy
)
print(f'Greedy: {tokenizer.decode(greedy_output[0], skip_special_tokens=True

# Top-p sampling with temperature
for temp in [0.3, 0.7, 1.2]:
    sampled = model.generate(
        input_ids, max_new_tokens=30,
        do_sample=True,
        temperature=temp,
        top_p=0.9,
    )
    text = tokenizer.decode(sampled[0], skip_special_tokens=True)
    print(f'\nTemp={temp}: {text}')

print('\n💡 Temperature controls randomness:')
print('   Low (0.3): focused, repetitive — good for factual tasks')
```

```
print('   Med (0.7): balanced — default for most tasks')
print('   High (1.2): creative, diverse — good for brainstorming')
```

4. Pipelines — Zero-Code Inference

HuggingFace `pipeline()` wraps tokeniser + model + post-processing:

In [ ]:
```
# ── Text Generation Pipeline ──────────────────────────────────────
generator = pipeline('text-generation', model='gpt2', device=-1)  # device=-

result = generator(
    "The patient was diagnosed with",
    max_new_tokens=40,
    temperature=0.7,
    do_sample=True,
    num_return_sequences=2,
)

for i, r in enumerate(result):
    print(f'\nGeneration {i+1}: {r["generated_text"]}')
```

In [ ]:
```
# ── Sentiment Analysis Pipeline ──────────────────────────────────
sentiment = pipeline('sentiment-analysis', device=-1)

texts = [
    "Patient shows significant improvement after therapy",
    "Experiencing severe anxiety and panic attacks daily",
    "Mood has been stable since medication adjustment",
]

for text in texts:
    result = sentiment(text)[0]
    print(f'{result["label"]:>8} ({result["score"]:.3f}): {text}')

print('\n💡 This uses a BERT-based model fine-tuned on sentiment data.')
print('   Same concept as what we\'ll do: fine-tune Phi-3 on mental health c
```

5. The `datasets` Library

HuggingFace `datasets` provides efficient data loading with:

- Memory mapping (handle datasets larger than RAM)
- Built-in train/test splits
- `.map()` for parallel preprocessing

In [ ]:
```
# ── Working with HF Datasets ──────────────────────────────────────
from datasets import Dataset
import pandas as pd

# Load our mental health CSV as a HuggingFace Dataset
```

```
# (In the real project, we do this in src/data/__init__.py)
try:
    df = pd.read_csv('../mental_health_dataset.csv')
    ds = Dataset.from_pandas(df)

    print(f'Dataset: {ds}')
    print(f'Columns: {ds.column_names}')
    print(f'First row: {ds[0]}')

    # Split
    split = ds.train_test_split(test_size=0.2, seed=42)
    print(f'\nTrain: {len(split["train"])} samples')
    print(f'Test: {len(split["test"])} samples')

    # Map function (apply preprocessing to all rows)
    def add_risk_label(example):
        dep = example.get('Depression Score', example.get('Depression_Score'
        anx = example.get('Anxiety Score', example.get('Anxiety_Score', 0))
        if isinstance(dep, (int, float)) and isinstance(anx, (int, float)):
            avg = (dep + anx) / 2
            example['risk_level'] = 'High' if avg > 0.6 else 'Moderate' if a
        else:
            example['risk_level'] = 'Unknown'
        return example

    ds_with_risk = split['train'].map(add_risk_label)
    print(f'\nWith risk labels: {ds_with_risk.column_names}')
except FileNotFoundError:
    print('mental_health_dataset.csv not found — using synthetic data instea
    data = {
        'text': ['Patient feels anxious', 'Mood is improving', 'Severe depre
        'label': [1, 0, 1]
    }
    ds = Dataset.from_dict(data)
    print(f'Synthetic dataset: {ds}')
```

6. Understanding Model Outputs

What exactly does a model return? Let's look under the hood:

In [ ]:

```
# ── Model outputs explained ──────────────────────────────────────

prompt = "Mental health assessment indicates"
inputs = tokenizer(prompt, return_tensors='pt')

with torch.no_grad():
    outputs = model(**inputs, output_hidden_states=True, output_attentions=T

print('Model output keys:', list(outputs.keys()))
print(f'\nlogits: {outputs.logits.shape}')  # (batch, seq_len, vocab_size)
print(f'  → probability distribution over next token at each position')

print(f'\nhidden_states: {len(outputs.hidden_states)} layers')
for i, hs in enumerate(outputs.hidden_states[:3]):
```

```
    print(f'  Layer {i}: {hs.shape}')  # (batch, seq_len, d_model)
print(f'  ...')
print(f'  Layer {len(outputs.hidden_states)-1}: {outputs.hidden_states[-1].s

print(f'\nattentions: {len(outputs.attentions)} layers')
print(f'  Layer 0: {outputs.attentions[0].shape}')  # (batch, n_heads, seq,

print(f'\n💡 hidden_states[-1] is the final representation of each token.')
print('   This is what gets passed to the output head for next-token predict
print('   Fine-tuning adjusts these representations for your specific task.'
```

7. Chat/Instruction Format

Modern fine-tuned models (Phi-3-instruct, LLaMA-3-chat) expect a specific prompt format. This is critical for our fine-tuning project.

Phi-3 Chat Template:

```
<|user|>
Assess this patient: Age 35, anxiety score 0.8, depression
score 0.6...
<|end|>
<|assistant|>
Severity: High

Based on the clinical indicators...
<|end|>
```

Why format matters:

The base model was pre-trained with these special tokens. If you fine-tune with a different format, the model gets confused. Always match the model's expected chat template.

In [ ]:
```
# ─── Chat template demonstration ──────────────────────────────────

# GPT-2 doesn't have a chat template, so let's show the concept
# (In Notebook 09-10, we'll use Phi-3's actual template)

def format_phi3_prompt(user_message, assistant_response=None):
    """
    Format a prompt for Phi-3 instruct model.
    This is what our src/data/prompt_formatter.py does.
    """
    prompt = f"<|user|>\n{user_message}\n<|end|>\n<|assistant|>\n"
    if assistant_response:
        prompt += f"{assistant_response}\n<|end|>"
    return prompt

# Example of what our fine-tuning data looks like
user_msg = """Assess this patient:
```

```
- Age: 34, Female
- Symptoms: Persistent low mood, fatigue, loss of interest
- Therapy History: 6 months CBT
- Depression Score: 0.72
- Anxiety Score: 0.45"""

assistant_msg = """Severity: High

Based on the clinical profile, this 34-year-old female presents with
elevated depression indicators (0.72) alongside moderate anxiety (0.45).
The persistent symptoms despite 6 months of CBT suggest treatment-resistant
features. Recommend psychiatric evaluation for medication adjustment."""

print('=== Training Example (what the model learns from) ===')
print(format_phi3_prompt(user_msg, assistant_msg))
print('\n💡 During training, the model learns to generate the assistant part
print('   given the user part. The prompt tokens are masked (loss = -100).')
```

✅ Key Takeaways

1. **AutoTokenizer**: handles BPE tokenisation automatically for any model
2. **AutoModelForCausalLM**: loads any decoder-only model (GPT, LLaMA, Phi)
3. **pipeline()**: easiest way to run inference (text-gen, sentiment, etc.)
4. **datasets library**: efficient data loading with `.map()` for preprocessing
5. **Generation parameters**: temperature, top_p, top_k control output diversity
6. **Chat templates**: critical for instruction-tuned models — match the format!

Connecting to Our Project

```
src/data/prompt_formatter.py  → formats data into Phi-3 chat
template
src/model/__init__.py         → loads model with
AutoModelForCausalLM
scripts/train.py              → uses Trainer from
transformers
```

Next: 09 — Fine-Tuning Approaches — WHY and HOW we adapt pre-trained models